

How Computers Work · Section 7

Files and File Systems

By TOUCH DEN · November 11, 2026

Contents

What You Will Learn 1

Before You Start 1

1. What a File Really Is 1

2. File Systems 2

3. Paths and Directories 2

4. Metadata and Permissions 3

5. File Types and Magic Bytes, and Why the Extension Can Lie 3

6. Deleting vs. Really Deleting 4

7. Mounting and Devices 5

Common Mistakes 5

Summary 6

Check Your Understanding 6

Practice 7

What You Will Learn

- What a file really is.
- What a file system is, and what it does.
- Paths and directories.
- File metadata and permissions.
- File types and magic bytes, and why the extension can lie.
- The difference between deleting and *really* deleting a file.
- Mounting, and how devices appear as part of the file system.

Before You Start

Read lessons 2, 4, and 6. You can read a hex dump, you know storage is persistent, and you know processes work in memory. This lesson is about how data rests on disk when a program is *not* running. The console examples use `file`, `xxd`, and `ls`, shown from real runs (with a neutral owner name).

---

1. What a File Really Is

You think of a file as a document, a photo, a song. Underneath, a **file is just a named sequence of bytes stored on a drive.** That is the whole definition. A photo, a program, a video, and a text note are all the same kind of thing to the computer: a string of bytes with a name. What makes one a photo and another a program is only *how the bytes are arranged and how they are meant to be read*, which is the lesson-1 idea (numbers mean nothing until something decides how to read them) applied to storage.

This simple truth has real consequences:

- **Any file can be examined as raw bytes** with a hex dump (lesson 2), regardless of what kind of file it is. That is how reverse engineers and forensic analysts look *inside* files.
- **The "type" of a file is not a deep property of the bytes;** it is a convention about how to read them. We will see in topic 5 that the type can be faked.

So when you handle a file in security, remember what it actually is: bytes with a name. Everything else, "it's a PDF", "it's an image", is interpretation.

2. File Systems

A drive (lesson 4) is just an enormous space for bytes. Something has to organize that space, keep track of which bytes belong to which file, where each file starts, what it is called, and who may read it. That organizer is the **file system**: the method the OS uses to store and find files on a drive.

You have met file system names even if you did not know it: **NTFS** (Windows), **APFS** (modern macOS), **ext4** (Linux), **FAT32** and **exFAT** (USB sticks and SD cards). Each is a different scheme for laying out files on storage, but they all do the same core jobs:

- Keep a **directory** (an index) of every file: its name, where its bytes are, and its size.
- Track which parts of the drive are **used** and which are **free**.
- Store each file's **metadata**: its permissions, timestamps, and more (topic 4).

The crucial thing to understand is that **a file's name and its bytes are stored separately.** The file system keeps a record ("the file `menu.txt` lives in these locations on the drive") apart from the actual bytes. This separation seems like a detail, but it is the key to understanding deletion (topic 6) and much of disk forensics. For now: the file system is the drive's filing clerk, keeping an index of where everything is.

3. Paths and Directories

Files are organized into **directories** (also called **folders**), and directories can contain other directories, forming a tree. To name a file's exact location in that tree, you write a **path**: the sequence of directories leading to it, separated by slashes.

- On Linux and macOS: `/home/user/documents/menu.txt` (forward slashes, starting from the root `/`).
- On Windows: `C:\Users\user\Documents\menu.txt` (backslashes, starting from a drive letter).

A path starting from the very top (the **root**) is an **absolute path**, it works from anywhere. A path starting from where you currently are is a **relative path**. Two special names appear in relative paths: `.` means "the current directory" and `..` means "the parent directory, one level up."

That `..` is small but important in security. Because `..` means "go up a level," a carelessly handled path can be tricked into climbing *out* of where it should stay. An attacker who supplies a filename like `../../etc/passwd` may escape the intended folder and reach sensitive files. This is a real and common web attack called **path traversal** (or directory traversal), studied in Web Application Security. The lesson here: paths are not just locations, they are untrusted input when they come from a user, and `..` is the classic trick.

4. Metadata and Permissions

Besides its bytes, every file carries **metadata**: data *about* the file, kept by the file system. Metadata includes the file's size, its timestamps (when it was created, last modified, last accessed), and its **permissions** (who may do what with it).

Permissions are where this connects straight back to Security Fundamentals and lesson 5. On Linux and macOS, every file has permissions for three kinds of people, the **owner**, the **group**, and **everyone else**, and three kinds of action, **read (r)**, **write (w)**, and **execute (x)**. The `ls -l` command shows them:

```
$ ls -l
-rw-r-----  user  staff  menu.txt
-rw-r--r--  user  staff  dot.png
```

Read the first column, `-rw-r-----`, in groups: the first character is the type (`-` for a normal file, `d` for a directory), then three characters for the **owner** (`rw-`: read and write), three for the **group** (`r--`: read only), and three for **everyone else** (`---`: nothing). So `menu.txt` can be read and written by its owner, read by its group, and not touched by anyone else, exactly the least-privilege idea (Security Fundamentals, lesson 7), enforced per file by the OS.

Metadata matters enormously in digital forensics (course 28). Timestamps build a **timeline** of what happened and when (did this file appear right after the suspicious login?). Permissions reveal what an attacker could reach. And because metadata can sometimes be altered, investigators treat it carefully and cross-check it. For now: a file is its bytes *plus* its metadata, and the permissions in that metadata are how the OS controls access to it.

5. File Types and Magic Bytes, and Why the Extension Can Lie

How does a computer know a file is a PNG image and not a text document? You might say "the extension, `.png`." But the extension is just part of the **name**, a hint, and anyone can change it. Rename `photo.png` to `photo.txt` and the bytes do not change at all. So the extension cannot be trusted.

The real answer is usually inside the file. Most file formats begin with a few fixed, recognizable bytes called a **magic number** or **magic bytes**, a signature that identifies the format. A PNG image always starts with the bytes `89 50 4e 47` (the `50 4e 47` spells "PNG" in ASCII, from lesson 3). Look:

```
$ xxd dot.png | head -1
00000000: 8950 4e47 0d0a 1a0a 0000 000d 4948 4452  .PNG........IHDR
```

There it is: `8950 4e47` at the very start, the PNG signature, whatever the file is named. Tools identify files by these signatures, not by the extension. The `file` command does exactly that, reading the content:

```
$ file menu.txt dot.png
menu.txt: ASCII text
dot.png:  PNG image data, 1 x 1, 8-bit gray+alpha, non-interlaced
```

Notice `file` correctly calls `dot.png` a PNG image by looking at its bytes, not its name. This gap, extension versus real content, is a security goldmine:

- **Attackers hide malware by faking the extension.** A file named `invoice.pdf` might really be a program; a photo-looking file might carry hidden data. Checking the *real* type (by magic bytes) instead of trusting the name is a basic defensive and forensic skill.
- **File-upload filters that trust the extension are bypassable.** If a website only checks that a filename ends in `.jpg`, an attacker uploads a program renamed to `.jpg`. Real checks look at the content (Web Application Security covers this).

So: the extension is a changeable hint; the magic bytes are the evidence. Trust the content, not the name.

6. Deleting vs. Really Deleting

When you delete a file and empty the trash, is it gone? Usually **not**, and understanding why is one of the most important ideas in forensics. Recall from topic 2 that a file's *name/index entry* and its *actual bytes* are stored separately.

When you "delete" a file, the file system normally just **removes the index entry and marks that space as free.** The actual bytes are *not* erased, they sit on the drive exactly as before, until the space happens to be reused for another file. It is like removing a book's card from the library catalogue but leaving the book on the shelf: the catalogue says the shelf is empty, but the book is still physically there until someone puts a new book in its place.

This has two big consequences:

- **Deleted files can often be recovered.** Forensic tools scan the drive for the leftover bytes of files whose index entries are gone, a technique called **file carving** (often using magic bytes from topic 5 to spot where files begin). This is how investigators recover files a suspect "deleted", a core part of digital forensics.
- **Truly erasing data takes extra work.** To actually destroy a file's contents, you must *overwrite* the bytes (or encrypt the whole drive so leftover bytes are meaningless, or

physically destroy the drive). "Secure delete" and "wipe" tools do this. The Data Security course (33) covers secure deletion properly.

So "delete" usually means "forget where it is", not "destroy it". That is good news for an investigator recovering evidence, and a warning for anyone who thinks deleting a sensitive file makes it gone.

7. Mounting and Devices

A final idea that ties storage to the file system. A drive's file system does not appear by magic; the OS has to **mount** it: connect the file system on a device into the overall tree of folders you browse, so its files become reachable at some location.

On Windows, mounting a USB stick makes it appear as a new drive letter like `E:`. On Linux and macOS, a mounted drive appears as a folder somewhere in the single tree (for example under `/Volumes` on macOS or `/mnt` on Linux). Either way, mounting is the act of saying "this device's files are now available here."

Mounting matters in forensics and security in a specific, important way. When an investigator examines a suspect's drive, they mount it **read-only**, so that the act of looking cannot change anything on it, preserving the evidence (and often using a hardware **write blocker** to guarantee it). This connects to the forensic principle you will meet in course 28: never alter the original evidence. More broadly, understanding mounting demystifies where your files "come from": every drive, USB stick, and memory card is a file system that the OS mounts into the tree you see.

You now understand how data lives at rest: a file is bytes plus metadata, organized by a file system, named by a path, typed by its magic bytes, recoverable after deletion, and made available by mounting. Next, we look at a special and important kind of file, the **program**, and how source code becomes something the CPU can run.

Common Mistakes

1. Trusting the file extension. The extension is a changeable part of the name. The real type is in the content (magic bytes). Attackers fake extensions to hide malware.

2. Thinking "deleted" means "gone". Deleting usually just removes the index entry; the bytes remain and can be recovered. Secure deletion requires overwriting or encryption.

3. Forgetting that name and bytes are stored separately. This separation explains both deletion and forensic recovery. It is the key idea of the lesson.

4. Ignoring `..` in user-supplied paths. `..` means "up a level" and can be abused to escape a folder (path traversal). Paths from users are untrusted input.

5. Examining evidence on a writable mount. Looking at a drive can change it (timestamps, etc.). Forensics mounts read-only to preserve the original.

Summary

Term	Remember
File	A named sequence of bytes on a drive; "type" is interpretation
File system	The OS's scheme to store/find files (NTFS, APFS, ext4, FAT32); keeps an index
Name vs bytes	Stored separately, the key to deletion and recovery
Path	A file's location in the tree; `..` = up a level (path-traversal risk)
Metadata & permissions	Data about a file; `rwx` for owner/group/others, OS-enforced access
Magic bytes	A file's real type is in its opening signature, not its extension
Delete vs wipe	Delete removes the index (bytes remain, recoverable); wipe overwrites
Mount	Connecting a device's file system into the tree; forensics mounts read-only

Check Your Understanding

1. What, fundamentally, is a file?
2. What does a file system do, and why does it store a file's name separately from its bytes?
3. In `-rw-r-----`, what can the owner, the group, and everyone else each do?
4. Why can't you trust a file's extension to know its real type? What *can* you trust?
5. When you delete a file and empty the trash, what usually happens to its bytes, and why can it often be recovered?
6. What is `..` in a path, and what attack abuses it?
7. Why does a forensic investigator mount a suspect's drive read-only?

Answers

1. A named sequence of bytes stored on a drive. What makes it a photo or a program is only how those bytes are arranged and read.
2. It organizes a drive's space: it keeps an index of every file (name, location, size), tracks used/free space, and stores metadata. It stores name and bytes separately, which is what makes deletion and forensic recovery work the way they do.

3. Owner: read and write (`rw-`). Group: read only (`r--`). Everyone else: nothing (`---`).
4. Because the extension is just a changeable part of the name; renaming doesn't change the bytes. You can trust the content, especially the magic bytes at the start, which identify the real format (as the `file` command does).
5. The file system usually just removes the index entry and marks the space free; the actual bytes remain until overwritten. So tools can scan the drive and recover the leftover bytes (file carving).
6. `..` means the parent directory, one level up. Path traversal abuses it (e.g. `../../etc/passwd`) to escape the intended folder and reach other files.
7. So that the act of examining the drive cannot change anything on it, preserving the original evidence (never alter the original).

Practice

1. Rename a harmless photo on your computer from `.png` to `.txt` and open it in a text editor. What do you see, and what does this prove about extensions versus bytes? (Then rename it back.)
2. If you have a terminal, run `file` on a few different files (a photo, a text file, a PDF) and see what it reports. Does it match or ignore the extension?
3. Explain to a friend, using the library-catalogue analogy, why a file they "deleted" might still be recoverable, and what they would need to do to really destroy it.

A Worked Example

1. Opening the renamed photo in a text editor shows mostly garbage, with maybe a few readable fragments, because a text editor tries to read image bytes as text. It proves the extension is only a label: the bytes are still a PNG, so changing `.png` to `.txt` changed the name but not the content.
2. `file photo.png` reports "PNG image data", `file notes.txt` reports "ASCII text", `file report.pdf` reports "PDF document", each by reading the content's magic bytes, not by trusting the extension. (Rename one and `file` still reports the true type.)
3. "Deleting a file is like removing its card from the library catalogue but leaving the book on the shelf, the catalogue says the space is free, but the book is still there until a new book replaces it, so someone can still find it. To really destroy it, you'd overwrite the space (a secure-wipe tool) or encrypt the whole drive so the leftover bytes are meaningless."

Course mark icon from Material Symbols by Google, Apache License 2.0.